

Intro to OOP — Objects & Classes

  RamRodz Robotics · Team 6366 · Programming Worksheet



Name _____

Date _____

  **RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS · SIMPSONVILLE, SC**

 **Focus Check-In — How are you feeling right now?**



Struggling



Okay



Ready



TODAY'S BIG QUESTION

WPILib gives us a motor, a controller, and a drivetrain — each one does different things. How does the code keep all of that organized without turning into a giant mess?

P Parking Lot — Random Thoughts

If your brain jumps to a question or idea, write it here so you don't lose it — ask later!

1 Key Words — Check Off When Explained



Class

A blueprint that describes what an object looks like and what it can do — like a recipe before the food is made.

Object

A real thing built from a class blueprint. You can have many objects from one class — just like many cars built from the same design.

Field

A variable that lives inside a class and holds data the object remembers — like a robot part knowing its own speed.

Constructor

A special method that runs once when you create an object — it sets the object's starting values.

Instance

Another word for "one specific object." If you make two Shooter objects, each one is a separate instance.

Dot Notation

The way you talk to an object: `objectName.methodName()` — like telling a specific robot part what to do.

2 The Blueprint — What is a Class?



Think about it: A car factory has one set of blueprints, but it can build thousands of cars from it. A **class** is like that blueprint — you write it once, and Java can create as many objects from it as you need.



Listen — A class is a *description*, not a thing yet. It has no memory and does nothing until an object is created from it.

ANATOMY OF A CLASS

```
class Shooter {           // class name – always capitalized

    // fields go here (what the object remembers)
    // constructor goes here (how to set it up)
    // methods go here (what the object can do)

}
```

 Plain English: The word **class** starts the blueprint. The curly braces { } hold everything that belongs to it. Nothing happens yet — this is just the description.

 **Quick Question — In one word, what is a class?**

3 The Real Thing — What is an Object?



☐ Listen — An object is a *real instance* built from the class blueprint. You create one using the new keyword.

Java — Creating an Object

```
Shooter myShooter = new Shooter(); // build a real Shooter object
```

 Plain English: **new Shooter()** takes the blueprint and builds a real object in memory. Now myShooter is a real thing — it has its own data and can do things.

 **Key idea:** You can have multiple objects from one class.

```
Shooter frontShooter = new Shooter();
```

```
Shooter backShooter = new Shooter();
```

Both are Shooters, but they are totally separate — changing one doesn't change the other.

 **Try It — Write a line that creates a new Shooter object named topShooter**

4 Fields — What an Object Remembers

- ☐ Listen — Fields are variables that live *inside* the class. Every object gets its own copy of each field.

Java — Adding Fields to a Class

```
class Shooter {  
  
    double speed;        // how fast the shooter spins (0.0 to 1.0)  
    boolean isRunning; // is the shooter currently on?  
  
}
```

 Plain English: **Fields are the object's memory.** The Shooter object will remember its own speed and whether it's isRunning. Each Shooter you create gets its own copy of these.

 **Think It — What two things does a Shooter object remember, according to the code above?**

5 The Constructor — Setting Up a New Object

- ☐ Listen — The constructor runs automatically when you use new. It gives the object its starting values so it's ready to use.

Java — Shooter Class with Constructor

```
class Shooter {  
  
    double speed;      // field: spin speed  
    boolean isRunning; // field: on or off  
  
    Shooter() {        // constructor — same name as the class, no re  
        speed = 0.0;    // start stopped  
        isRunning = false; // start off  
    }  
  
}
```

💡 Plain English: **The constructor is like the "setup" step.** The moment you write new Shooter(), Java runs the constructor and sets speed to 0 and isRunning to false — so the shooter always starts in a safe, stopped state.

✏ **Quick Question — When does the constructor run?**

6 Dot Notation — Talking to Your Object

- ☐ Listen — After you create an object, you use a *dot* to call its methods: `objectName.methodName()`. This is called dot notation.

```

class Shooter {

    double speed;
    boolean isRunning;

    Shooter() {
        speed = 0.0;
        isRunning = false;
    }

    void spinUp() {          // method: start the shooter
        isRunning = true;
        speed = 0.8;         // 80% power
    }

    void stop() {            // method: stop the shooter
        isRunning = false;
        speed = 0.0;         // no power
    }

}

// Using the class:
Shooter myShooter = new Shooter(); // create the object
myShooter.spinUp();                 // tell it to spin up
myShooter.stop();                   // tell it to stop

```

💡 Plain English: **The dot is how you give an order to a specific object.**

`myShooter.spinUp()` means "hey, *myShooter*, run your `spinUp` method." If you had two shooters, each one responds only to its own dot.

🔧 **WPILib works exactly the same way!**

```

PWMSparkMax motor = new PWMSparkMax(0); ← create a motor object
motor.set(0.5); ← dot notation to set its speed to 50%

```

Every WPILib component — motors, controllers, encoders — is an **object** built from a **class**.

 **Try It — Write the dot-notation call to stop the shooter using the myShooter object**

Robot Scenario — Design a Drivetrain Class

The Challenge

Your robot needs a Drivetrain class to control its two drive motors. The drivetrain should:

- Remember the current speed for the **left** motor and the **right** motor (both start at 0.0)
- Have a method called `drive()` that could set both motors to full power
- Have a method called `stop()` that sets both speeds back to 0.0

☐ I understand the challenge — a Drivetrain class needs fields for left and right speed, a constructor to set them to 0, and at least two methods.

 **Sketch It — Write out the skeleton of a Drivetrain class (fields + constructor + method names)**

 **Apply It — Write the line that creates a Drivetrain object named dt and calls its `drive()` method**

✓ **End of Lesson — I Can...**

- ☐ Explain the difference between a **class** (blueprint) and an **object** (real thing built from it).
- ☐ Write a basic class in Java with at least two **fields** and a **constructor** that sets their starting values.
- ☐ Use **new** to create an object from a class and store it in a variable.
- ☐ Use **dot notation** to call a method on an object: `myShooter.spinUp()`.
- ☐ Recognize that WPILib components (motors, controllers) are **objects** — and that `motor.set(0.5)` is dot notation calling a method.